

Sistema de Control de Órdenes de Servicio de un Taller Automotriz

Andrés Morataya

26497

Análisis del Sistema (30 puntos)

1.1 Requisitos funcionales del sistema (5 pts)

Escribir los puntos exactos que debe cumplir tu sistema.

- El usuario puede registrar una nueva orden con número, propietario, placa, descripción y costo estimado.
- El sistema almacena una cantidad dinámica de órdenes mediante una colección declarada como List<Orden> e inicializada con ArrayList<Orden>.
- El sistema valida que el número de orden no esté vacío ni repetido.
- El sistema valida que el nombre del propietario, la placa y la descripción no estén vacíos.
- El sistema valida que el costo estimado sea mayor que cero.
- El usuario puede consultar todas las órdenes registradas y recibir un aviso si no existen órdenes.
- El usuario puede buscar una orden por su número y visualizar toda su información.
- El sistema informa mediante una excepción controlada cuando una orden buscada no existe y continúa funcionando.
- El usuario puede modificar la descripción y el costo estimado de una orden existente.
- El usuario puede cancelar una orden existente, eliminándola de la colección dinámica.
- El usuario puede consultar todas las órdenes asociadas a una placa, incluyendo varias coincidencias.
- El sistema calcula y muestra el costo total de las órdenes activas.
- El sistema calcula y muestra el costo promedio de las órdenes activas; si no hay órdenes, devuelve 0.

- El sistema identifica y muestra la orden con el costo estimado más alto.
- El sistema muestra la cantidad actual de órdenes registradas.
- El programa controla entradas numéricas incorrectas mediante try-catch, sin finalizar inesperadamente.
- El programa controla mediante try-catch las búsquedas, modificaciones o cancelaciones de órdenes inexistentes.
- El programa ejecuta un bloque finally al terminar cada operación y permite regresar al menú principal.
- El usuario puede seleccionar la opción Salir para terminar el programa y cerrar el Scanner.

1.2 Clases necesarias y su propósito (5 pts)

Clase	Propósito
Orden	Representa una orden de servicio y encapsula los datos del propietario, vehículo, servicio y costo.
Taller	Administra la colección dinámica de órdenes y concentra las operaciones de registro, consulta, búsqueda, modificación, cancelación y reportes.
Ordenfallida	Excepción comprobada personalizada que comunica operaciones inválidas relacionadas con órdenes duplicadas, inexistentes o con una colección vacía.
Main	Driver program. Presenta el menú, captura datos, invoca los métodos de Taller y controla las excepciones para mantener el programa en ejecución.

1.3 Atributos de cada clase (10 pts)

Clase: Orden

Atributo	Tipo de dato	Visibilidad	Propósito
norden	String	- private	Identificador de la orden; permite localizarla y comprobar que no esté duplicada.
nombre	String	- private	Nombre del propietario del vehículo.
placa	String	- private	Placa del vehículo; se usa para agrupar y consultar sus órdenes.
descripcion	String	- private	Descripción del mantenimiento o reparación solicitada.
costo	double	- private	Costo estimado del servicio; participa en los reportes de costos.

Clase: Taller

Atributo	Tipo de dato	Visibilidad	Propósito
ordenes	List<Orden>	- private	Colección dinámica que almacena las órdenes activas. Se declara con la interfaz List y se inicializa mediante new ArrayList<>().

Clase: Ordenfallida

Atributo	Tipo de dato	Visibilidad	Propósito
No declara atributos propios	—	—	Hereda de Exception el estado necesario, incluido el mensaje proporcionado al constructor.

Clase: Main

Atributo	Tipo de dato	Visibilidad	Propósito
No declara atributos de clase	—	—	scanner, taller, opcion, numero, nombre, placa, descripcion y costo son variables locales del método main; por ello no son atributos UML de Main.

1.4 Métodos de cada clase (10 pts)

Clase: Orden

Método	Parámetros: tipo de dato	Resultado / retorno	Visibilidad	Propósito y excepciones
Orden	Ninguno	— (constructor)	+ public	Crea una orden sin asignar valores explícitos; Java deja String en null y double en 0.0.
Orden	norden: String; nombre: String; placa: String; descripcion: String; costo: double	— (constructor)	+ public	Inicializa una orden con toda la información recibida.
getnorden	Ninguno	String	+ public	Devuelve el número de orden.
getnombre	Ninguno	String	+ public	Devuelve el nombre del propietario.
getplaca	Ninguno	String	+ public	Devuelve la placa del vehículo.
getdescripcion	Ninguno	String	+ public	Devuelve la descripción del servicio.
getcosto	Ninguno	double	+ public	Devuelve el costo estimado.
setdescripcion	descripcion: String	void	+ public	Actualiza la descripción de la orden.
setcosto	costo: double	void	+ public	Actualiza el costo estimado de la orden.
toString	Ninguno	String	+ public	Devuelve todos los datos de la orden en un formato legible.
equals	obj: Object	boolean	+ public	Compara dos órdenes por todos sus atributos y devuelve true si son iguales.

Clase: Taller

Método	Parámetros: tipo de dato	Resultado / retorno	Visibilidad	Propósito y excepciones
Taller	Ninguno	— (constructor)	+ public	Inicializa ordenes como una colección vacía: <code>new ArrayList<>()</code> .
registrarorden	norden: String; nombre: String; placa: String; descripcion: String; costo: double	void	+ public	Valida los datos, evita números duplicados, crea la Orden y la agrega. Lanza <code>Ordenfallida</code> o <code>IllegalArgumentException</code> .
existenciaorden	newnorden: String	boolean	+ public	Recorre la colección y comprueba, sin distinguir mayúsculas, si ya existe el número.
consultarordenes	Ninguno	List<Orden>	+ public	Devuelve una copia de la lista para consultar las órdenes sin exponer directamente la colección interna.
buscarorden	newnorden: String	Orden	+ public	Busca por número sin distinguir mayúsculas; lanza <code>Ordenfallida</code> si no encuentra coincidencia.
cancelarorden	newnorden: String	void	+ public	Busca por índice y elimina la orden de la colección; lanza <code>Ordenfallida</code> si no existe.
coincidenciasplaca	placa: String	List<Orden>	+ public	Devuelve una nueva lista con todas las órdenes cuya placa coincide sin distinguir mayúsculas.
totalordenes	Ninguno	double	+ public	Recorre la colección y suma los costos estimados.
promedio	Ninguno	double	+ public	Devuelve 0 si la lista está vacía; de lo contrario divide el total entre la cantidad de órdenes.
costomasalto	Ninguno	Orden	+ public	Recorre las órdenes y devuelve la de mayor costo; lanza <code>Ordenfallida</code>

				si no hay órdenes.
cantidadordenesregistradas	Ninguno	int	+ public	Devuelve el tamaño actual de la colección.
modificarorden	norden: String; descripcion: String; costo: double	void	+ public	Valida los nuevos datos, busca la orden y actualiza descripción y costo. Puede lanzar Ordenfallida o IllegalArgumentException.

Clase: Ordenfallida

Método	Parámetros: tipo de dato	Resultado / retorno	Visibilidad	Propósito y excepciones
Ordenfallida	mensaje: String	— (constructor)	+ public	Crea la excepción personalizada y envía el mensaje al constructor de Exception mediante super(mensaje).

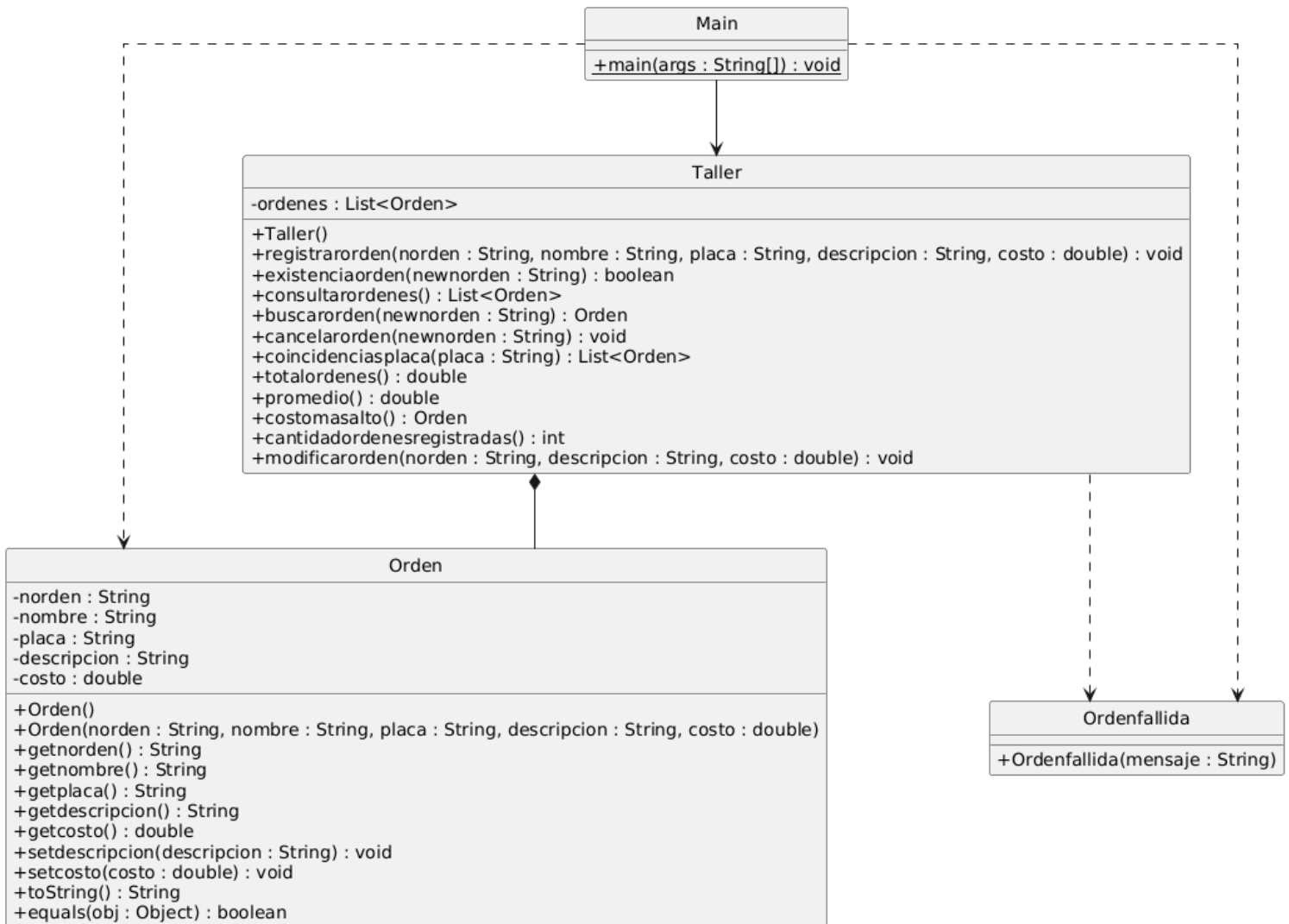
Clase: Main

Método	Parámetros: tipo de dato	Resultado / retorno	Visibilidad	Propósito y excepciones
main	args: String[]	void	+ public static	Inicia el sistema, crea Scanner y Taller, muestra el menú de 10 opciones, procesa la selección con un ciclo do-while, controla NumberFormatException, Ordenfallida e IllegalArgumentException ,ejecuta finally y cierra el Scanner al salir.

El bloque finally se utilizará para ejecutar una acción de cierre de cada operación y permitir la continuidad hacia el menú principal. Esta acción debe ejecutarse independientemente de si la operación terminó correctamente o produjo una excepción.

2. Diseño: Diagrama de Clases (30 puntos)

- Asegúrate de mostrar atributos y métodos con visibilidad (+, -).
- Indica relaciones entre clases (asociación, agregación, etc.).



3. Programa (40 puntos)

En cada archivo `.java`, asegurarse de incluir:

- Las clases necesarias.
- Uso adecuado de objetos.

Menú que debe implementar el driver program:

1. Registrar orden
2. Consultar órdenes
3. Buscar orden
4. Modificar orden
5. Cancelar orden
6. Consultar órdenes por placa
7. Reporte de costos
8. Orden de mayor costo
9. Cantidad de órdenes
10. Salir

GitHub: colocar aquí la URL:

<https://github.com/AndresMorataya0/Ejercicio3-Arreglos-Dinamicos.git>
antes de entregar

- ☒ Está claro el análisis?
- ☒ El diagrama tiene los elementos UML correctamente?
- ☒ Subiste tu código a GitHub con todo lo necesario?